

EXTEND WITH TOOLS

Give Claude custom tools

Define custom tools with the Claude Agent SDK's in-process MCP server so Claude can call your functions, hit your APIs, and perform domain-specific operations.

Custom tools extend the Agent SDK by letting you define your own functions that Claude can call during a conversation. Using the SDK's in-process MCP server, you can give Claude access to databases, external APIs, domain-specific logic, or any other capability your application needs.

This guide covers how to define tools with input schemas and handlers, bundle them into an MCP server, pass them to `query`, and control which tools Claude can access. It also covers error handling, tool annotations, and returning non-text content like images.

Quick reference

If you want to...	Do this
Define a tool	Use <code>@tool</code> (Python) or <code>tool()</code> (TypeScript) with a name, description, schema, and handler. See Create a custom tool .
Register a tool with Claude	Wrap in <code>create_sdk_mcp_server</code> / <code>createSdkMcpServer</code> and pass to <code>mcpServers</code> in <code>query()</code> . See Call a custom tool .
Pre-approve a tool	Add to your allowed tools. See Configure allowed tools .
Remove a built-in tool from Claude's context	Pass a <code>tools</code> array listing only the built-ins you want. See Configure allowed tools .
Let Claude call tools in parallel	Set <code>readOnlyHint: true</code> on tools with no side effects. See Add tool annotations .
Handle errors without stopping the loop	Return <code>isError: true</code> instead of throwing. See Handle errors .
Return images or files	Use <code>image</code> or <code>resource</code> blocks in the content array. See Return images and resources .
Return a machine-readable JSON result	Set <code>structuredContent</code> on the result. See Return structured data .
Scale to many tools	Use tool search to load tools on demand.

Create a custom tool

A tool is defined by four parts, passed as arguments to the `tool()` helper in TypeScript or the `@tool` decorator in Python:

- **Name:** a unique identifier Claude uses to call the tool.
- **Description:** what the tool does. Claude reads this to decide when to call it.
- **Input schema:** the arguments Claude must provide. In TypeScript this is always a [Zod schema](#), and the handler's `args` are typed from it automatically. In Python this is a dict mapping names to types, like `{"latitude": float}`, which the SDK converts to JSON Schema for you. The Python decorator also accepts a full [JSON Schema](#) dict directly when you need enums, ranges, optional fields, or nested objects.
- **Handler:** the async function that runs when Claude calls the tool. It receives the validated arguments and must return an object with:
 - `content` (required): an array of result blocks, each with a `type` of `"text"`, `"image"`,

`"audio"` , `"resource"` , or `"resource_link"` . See [Return images and resources](#) for non-text blocks.

- `structuredContent` (optional): a JSON object holding the result as machine-readable data, returned alongside `content` . See [Return structured data](#).
- `isError` (optional): set to `true` to signal a tool failure so Claude can react to it. See [Handle errors](#).

After defining a tool, wrap it in a server with [createSdkMcpServer](#) (TypeScript) or [create_sdk_mcp_server](#) (Python). The server runs in-process inside your application, not as a separate process.

Weather tool example

This example defines a `get_temperature` tool and wraps it in an MCP server. It only sets up the tool; to pass it to `query` and run it, see [Call a custom tool](#) below.

```

from typing import Any
import httpx
from claude_agent_sdk import tool, create_sdk_mcp_server

# Define a tool: name, description, input schema, handler
@tool(
    "get_temperature",
    "Get the current temperature at a location",
    {"latitude": float, "longitude": float},
)
async def get_temperature(args: dict[str, Any]) -> dict[str, Any]:
    async with httpx.AsyncClient() as client:
        response = await client.get(
            "https://api.open-meteo.com/v1/forecast",
            params={
                "latitude": args["latitude"],
                "longitude": args["longitude"],
                "current": "temperature_2m",
                "temperature_unit": "fahrenheit",
            },
        )
        data = response.json()

    # Return a content array - Claude sees this as the tool result
    return {
        "content": [
            {
                "type": "text",
                "text": f"Temperature: {data['current']['temperature_2m']}°F",
            }
        ]
    }

# Wrap the tool in an in-process MCP server
weather_server = create_sdk_mcp_server(
    name="weather",
    version="1.0.0",
    tools=[get_temperature],
)

```

See the [tool\(\)](#) TypeScript reference or the [@tool](#) Python reference for full parameter details,

including JSON Schema input formats and return value structure.

💡 To make a parameter optional: in TypeScript, add `.default()` to the Zod field. In Python, the dict schema treats every key as required, so leave the parameter out of the schema, mention it in the description string, and read it with `args.get()` in the handler. The [get_precipitation_chance tool below](#) shows both patterns.

Call a custom tool

Pass the MCP server you created to `query` via the `mcpServers` option. The key in `mcpServers` becomes the `{server_name}` segment in each tool's fully qualified name:

`mcp__{server_name}__{tool_name}`. List that name in `allowedTools` so the tool runs without a permission prompt.

These snippets reuse the `weatherServer` from the [example above](#) to ask Claude what the weather is in a specific location.

```
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, ResultMessage

async def main():
    options = ClaudeAgentOptions(
        mcp_servers={"weather": weather_server},
        allowed_tools=["mcp__weather__get_temperature"],
    )

    async for message in query(
        prompt="What's the temperature in San Francisco?",
        options=options,
    ):
        # ResultMessage is the final message after all tool calls complete
        if isinstance(message, ResultMessage) and message.subtype ==
"success":
            print(message.result)

asyncio.run(main())
```

Add more tools

A server holds as many tools as you list in its `tools` array. With more than one tool on a server, you can list each one in `allowedTools` individually or use the wildcard `mcp__weather__*` to cover every tool the server exposes.

The example below adds a second tool, `get_precipitation_chance`, to the `weatherServer` from the [weather tool example](#) and rebuilds it with both tools in the array.

```

# Define a second tool for the same server
@tool(
    "get_precipitation_chance",
    "Get the hourly precipitation probability for a location. "
    "Optionally pass 'hours' (1-24) to control how many hours to return.",
    {"latitude": float, "longitude": float},
)
async def get_precipitation_chance(args: dict[str, Any]) -> dict[str, Any]:
    # 'hours' isn't in the schema - read it with .get() to make it optional
    hours = args.get("hours", 12)
    async with httpx.AsyncClient() as client:
        response = await client.get(
            "https://api.open-meteo.com/v1/forecast",
            params={
                "latitude": args["latitude"],
                "longitude": args["longitude"],
                "hourly": "precipitation_probability",
                "forecast_days": 1,
            },
        )
        data = response.json()
    chances = data["hourly"]["precipitation_probability"][ :hours]

    return {
        "content": [
            {
                "type": "text",
                "text": f"Next {hours} hours: {' '.join(map(str,
chances))}%".
            }
        ]
    }

# Rebuild the server with both tools in the array
weather_server = create_sdk_mcp_server(
    name="weather",
    version="1.0.0",
    tools=[get_temperature, get_precipitation_chance],
)

```

Every tool in this array consumes context window space on every turn. If you're defining dozens of tools, see [tool search](#) to load them on demand instead.

Add tool annotations

Tool annotations are optional metadata describing how a tool behaves. Pass them as the fifth argument to `tool()` helper in TypeScript or via the `annotations` keyword argument for the `@tool` decorator in Python. All hint fields are Booleans.

Field	Default	Meaning
<code>readOnlyHint</code>	<code>false</code>	Tool does not modify its environment. Controls whether the tool can be called in parallel with other read-only tools.
<code>destructiveHint</code>	<code>true</code>	Tool may perform destructive updates. Informational only.
<code>idempotentHint</code>	<code>false</code>	Repeated calls with the same arguments have no additional effect. Informational only.
<code>openWorldHint</code>	<code>true</code>	Tool reaches systems outside your process. Informational only.

Annotations are metadata, not enforcement. A tool marked `readOnlyHint: true` can still write to disk if that's what the handler does. Keep the annotation accurate to the handler.

This example adds `readOnlyHint` to the `get_temperature` tool from the [weather tool example](#).

```
from claude_agent_sdk import tool, ToolAnnotations

@tool(
    "get_temperature",
    "Get the current temperature at a location",
    {"latitude": float, "longitude": float},
    annotations=ToolAnnotations(
        readOnlyHint=True
    ), # Lets Claude batch this with other read-only calls
)
async def get_temperature(args):
    return {"content": [{"type": "text", "text": "..."}]}
```

See `ToolAnnotations` in the [TypeScript](#) or [Python](#) reference.

Control tool access

The [weather tool example](#) registered a server and listed tools in `allowedTools`. This section covers

how tool names are constructed and how to scope access when you have multiple tools or want to restrict built-ins.

Tool name format

When MCP tools are exposed to Claude, their names follow a specific format:

- Pattern: `mcp__{server_name}__{tool_name}`
- Example: A tool named `get_temperature` in server `weather` becomes `mcp__weather__get_temperature`

Configure allowed tools

The `tools` option and the allowed/disallowed lists affect two layers: availability, which controls whether a tool appears in Claude’s context, and permission, which controls whether a call is approved once Claude attempts it. `tools` and bare-name `disallowedTools` entries change availability. `allowedTools` and scoped `disallowedTools` rules change permission only.

Option	Layer	Effect
<code>tools: ["Read", "Grep"]</code>	Availability	Only the listed built-ins are in Claude’s context. Unlisted built-ins are removed. MCP tools are unaffected.
<code>tools: []</code>	Availability	All built-ins are removed. Claude can only use your MCP tools.
<code>allowed tools</code>	Permission	Listed tools run without a permission prompt. Unlisted tools remain available; calls go through the <u>permission flow</u> .
<code>disallowed tools</code>	Both	A bare tool name such as <code>"Bash"</code> removes the tool from Claude’s context, the same as omitting it from <code>tools</code> . A scoped rule such as <code>"Bash(rm *)"</code> leaves the tool in context and denies only matching calls.

To remove a built-in entirely, omit it from `tools` or list its bare name in `disallowedTools` (Python: `disallowed_tools`); both keep the tool out of context so Claude never attempts it. A scoped `disallowedTools` rule blocks matching calls but leaves the tool visible, so Claude may waste a turn trying it. See Configure permissions for the full evaluation order.

Handle errors

How your handler reports errors determines whether the agent loop continues or stops:

What happens	Result
Handler throws an uncaught exception	Agent loop stops. Claude never sees the error, and the <code>query</code> call fails.
Handler catches the error and returns <code>isError: true</code> (TS) / <code>"is_error": True</code> (Python)	Agent loop continues. Claude sees the error as data and can retry, try a different tool, or explain the failure.

The example below catches two kinds of failures inside the handler instead of letting them throw. A non-200 HTTP status is caught from the response and returned as an error result. A network error or invalid JSON is caught by the surrounding `try/except` (Python) or `try/catch` (TypeScript) and also returned as an error result. In both cases the handler returns normally and the agent loop continues.

```

import json
import httpx
from typing import Any

@tool(
    "fetch_data",
    "Fetch data from an API",
    {"endpoint": str}, # Simple schema
)
async def fetch_data(args: dict[str, Any]) -> dict[str, Any]:
    try:
        async with httpx.AsyncClient() as client:
            response = await client.get(args["endpoint"])
            if response.status_code != 200:
                # Return the failure as a tool result so Claude can react to
it.

                # is_error marks this as a failed call rather than odd-looking
data.

                return {
                    "content": [
                        {
                            "type": "text",
                            "text": f"API error: {response.status_code}
{response.reason_phrase}",
                        }
                    ],
                    "is_error": True,
                }

            data = response.json()
            return {"content": [{"type": "text", "text": json.dumps(data,
indent=2)}}]}
    except Exception as e:
        # Catching here keeps the agent loop alive. An uncaught exception
        # would end the whole query() call.
        return {
            "content": [{"type": "text", "text": f"Failed to fetch data:
{str(e)}"}],
            "is_error": True,
        }

```

Return images and resources

The `content` array in a tool result accepts `text` , `image` , `audio` , `resource` , and `resource_link` blocks. You can mix them in the same response. Audio blocks are saved to disk and Claude receives a text block with the saved file path. Resource link blocks are converted to a text block containing the link’s name, URI, and description.

Images

An image block carries the image bytes inline, encoded as base64. There is no URL field. To return an image that lives at a URL, fetch it in the handler, read the response bytes, and base64-encode them before returning. The result is processed as visual input.

Field	Type	Notes
<code>type</code>	<code>"image"</code>	
<code>data</code>	<code>string</code>	Base64-encoded bytes. Raw base64 only, no <code>data:image/...;base64,</code> prefix
<code>mimeType</code>	<code>string</code>	Required. For example <code>image/png</code> , <code>image/jpeg</code> , <code>image/webp</code> , <code>image/gif</code>

```

import base64
import httpx

# Define a tool that fetches an image from a URL and returns it to Claude
@tool("fetch_image", "Fetch an image from a URL and return it to Claude",
{"url": str})
async def fetch_image(args):
    async with httpx.AsyncClient() as client: # Fetch the image bytes
        response = await client.get(args["url"])

    return {
        "content": [
            {
                "type": "image",
                "data": base64.b64encode(response.content).decode(
                    "ascii"
                ), # Base64-encode the raw bytes
                "mimeType": response.headers.get(
                    "content-type", "image/png"
                ), # Read MIME type from the response
            }
        ]
    }

```

Resources

A resource block embeds a piece of content identified by a URI. The URI is a label for Claude to reference; the actual content rides in the block’s `text` or `blob` field. Use this when your tool produces something that makes sense to address by name later, such as a generated file or a record from an external system.

Field	Type	Notes
<code>type</code>	<code>"resource"</code>	
<code>resource.uri</code>	<code>string</code>	Identifier for the content. Any URI scheme
<code>resource.text</code>	<code>string</code>	The content, if it’s text. Provide this or <code>blob</code> , not both
<code>resource.blob</code>	<code>string</code>	The content base64-encoded, if it’s binary
<code>resource.mimeType</code>	<code>string</code>	Optional

This example shows a resource block returned from inside a tool handler. The URI

`file:///tmp/report.md` is a label that Claude can reference later; the SDK does not read from that path.

```
return {
  content: [
    {
      type: "resource",
      resource: {
        uri: "file:///tmp/report.md", // Label for Claude to reference, not a
path the SDK reads
        mimeType: "text/markdown",
        text: "# Report\n..." // The actual content, inline
      }
    }
  ]
};
```

These block shapes come from the MCP `CallToolResult` type. See the [MCP specification](#) for the full definition.

Return structured data

`structuredContent` is an optional JSON object on the result, separate from the `content` array. Use it to return raw values that Claude can read as exact fields instead of parsing them out of a text string or image.

When `structuredContent` is set, Claude receives the JSON plus any image or resource blocks from `content`. Text blocks in `content` are not forwarded, since they are assumed to duplicate the structured data. The example below renders a chart as an image block and returns the data points behind it in `structuredContent` from the same handler.

TypeScript

```
return {
  content: [
    {
      type: "image",
      data: chartPngBuffer.toString("base64"),
      mimeType: "image/png"
    }
  ],
  structuredContent: {
    series: "temperature_2m",
    unit: "fahrenheit",
    points: [62.1, 63.4, 65.0, 64.2]
  }
};
```

❗ The Python `@tool` decorator forwards only `content` and `is_error` from the handler's return dict. To return `structuredContent` from Python, run a [standalone MCP server](#) instead of an in-process SDK server.

Example: unit converter

This tool converts values between units of length, temperature, and weight. A user can ask “convert 100 kilometers to miles” or “what is 72°F in Celsius,” and Claude picks the right unit type and units from the request.

It demonstrates two patterns:

- **Enum schemas:** `unit_type` is constrained to a fixed set of values. In TypeScript, use `z.enum()`. In Python, the dict schema doesn't support enums, so the full JSON Schema dict is required.
- **Unsupported input handling:** when a conversion pair isn't found, the handler returns `isError: true` so Claude can tell the user what went wrong rather than treating a failure as a normal result.

```

from typing import Any
from claude_agent_sdk import tool, create_sdk_mcp_server

# z.enum() in TypeScript becomes an "enum" constraint in JSON Schema.
# The dict schema has no equivalent, so full JSON Schema is required.
@tool(
    "convert_units",
    "Convert a value from one unit to another",
    {
        "type": "object",
        "properties": {
            "unit_type": {
                "type": "string",
                "enum": ["length", "temperature", "weight"],
                "description": "Category of unit",
            },
            "from_unit": {
                "type": "string",
                "description": "Unit to convert from, e.g. kilometers,
fahrenheit, pounds",
            },
            "to_unit": {"type": "string", "description": "Unit to convert
to"},
            "value": {"type": "number", "description": "Value to convert"},
        },
        "required": ["unit_type", "from_unit", "to_unit", "value"],
    },
)
async def convert_units(args: dict[str, Any]) -> dict[str, Any]:
    conversions = {
        "length": {
            "kilometers_to_miles": lambda v: v * 0.621371,
            "miles_to_kilometers": lambda v: v * 1.60934,
            "meters_to_feet": lambda v: v * 3.28084,
            "feet_to_meters": lambda v: v * 0.3048,
        },
        "temperature": {
            "celsius_to_fahrenheit": lambda v: (v * 9) / 5 + 32,
            "fahrenheit_to_celsius": lambda v: (v - 32) * 5 / 9,
            "celsius_to_kelvin": lambda v: v + 273.15,
            "kelvin_to_celsius": lambda v: v - 273.15,
        },
        "weight": {

```



```

        "kilograms_to_pounds": lambda v: v * 2.20462,
        "pounds_to_kilograms": lambda v: v * 0.453592,
        "grams_to_ounces": lambda v: v * 0.035274,
        "ounces_to_grams": lambda v: v * 28.3495,
    },
}

key = f"{args['from_unit']}_to_{args['to_unit']}"
fn = conversions.get(args["unit_type"], {}).get(key)

if not fn:
    return {
        "content": [
            {
                "type": "text",
                "text": f"Unsupported conversion: {args['from_unit']} to
{args['to_unit']}",
            }
        ],
        "is_error": True,
    }

result = fn(args["value"])
return {
    "content": [
        {
            "type": "text",
            "text": f"{args['value']} {args['from_unit']} = {result:.4f}
{args['to_unit']}",
        }
    ]
}

converter_server = create_sdk_mcp_server(
    name="converter",
    version="1.0.0",
    tools=[convert_units],
)

```

Once the server is defined, pass it to `query` the same way as the weather example. This example sends three different prompts in a loop to show the same tool handling different unit types. For each response, it inspects `AssistantMessage` objects (which contain the tool calls Claude made during that turn) and prints each `ToolUseBlock` before printing the final `ResultMessage` text. This lets

you see when Claude is using the tool versus answering from its own knowledge.

```
import asyncio
from claude_agent_sdk import (
    query,
    ClaudeAgentOptions,
    ResultMessage,
    AssistantMessage,
    ToolUseBlock,
)

async def main():
    options = ClaudeAgentOptions(
        mcp_servers={"converter": converter_server},
        allowed_tools=["mcp__converter__convert_units"],
    )

    prompts = [
        "Convert 100 kilometers to miles.",
        "What is 72°F in Celsius?",
        "How many pounds is 5 kilograms?",
    ]

    for prompt in prompts:
        async for message in query(prompt=prompt, options=options):
            if isinstance(message, AssistantMessage):
                for block in message.content:
                    if isinstance(block, ToolUseBlock):
                        print(f"[tool call] {block.name}({block.input})")
                    elif isinstance(message, ResultMessage) and message.subtype ==
"success":
                        print(f"Q: {prompt}\nA: {message.result}\n")

asyncio.run(main())
```

Next steps

Custom tools wrap async functions in a standard interface. You can mix the patterns on this page in the same server: a single server can hold a database tool, an API gateway tool, and an image renderer alongside each other.

From here:

- If your server grows to dozens of tools, see [**tool search**](#) to defer loading them until Claude needs them.
- To connect to external MCP servers (filesystem, GitHub, Slack) instead of building your own, see [**Connect MCP servers**](#).
- To control which tools run automatically versus requiring approval, see [**Configure permissions**](#).

Related documentation

- [**TypeScript SDK Reference**](#)
- [**Python SDK Reference**](#)
- [**MCP Documentation**](#)
- [**SDK Overview**](#)